

Thibault KINE

Dossier de Projet Professionnel “Dkbldr”

Titre RNCP Concepteur Développeur d'Applications

Sommaire

Sommaire.....	3
Présentation du projet.....	6
Description.....	6
Périmètre du projet.....	7
Cibles adressées.....	7
Arborescence de l'application.....	8
Cahier des charges.....	10
Dénomination.....	11
Introduction.....	11
Contexte et Objectifs du Projet.....	11
Public Cible.....	11
Objectifs du Projet.....	11
Objectifs Généraux.....	11
Bénéfices Attendus.....	11
Spécifications Fonctionnelles.....	12
Gestion des decks.....	12
Outil de Recherche.....	12
Import/Export de decks.....	12
Spécifications Techniques.....	12
Technologies Utilisées.....	12
Contraintes Techniques.....	12
Descriptif des fonctionnalités.....	13
Authentification.....	14
Espace "profil".....	14
Construction de deck ("Deckbuilding").....	15
Deckbuilding.....	15
Outils & options.....	15
Page de recherche.....	16
Decks recommandés.....	16
Decks d'abonnements.....	16
"Commander of the Day".....	16
Spécifications techniques.....	18
Technologies back-end.....	19
Technologies front-end.....	21
Environnement de développement.....	23
Planification et Conception.....	23
Outils de développement.....	25
Contrôle de version.....	25

Conteneurisation.....	25
Pipeline CI/CD.....	25
Déploiement.....	26
Architecture du projet.....	27
Pourquoi cette architecture ?.....	28
Réalisation.....	30
Charte graphique.....	31
Maquette.....	31
Conception de la base de données.....	31
Authentification.....	33
Stockage de fichiers.....	33
Sources des icônes.....	34
Compétences attestées.....	35
Développer une application sécurisée :.....	36
Concevoir et développer une application sécurisée organisée en c.....	37
Préparer le déploiement d'une application sécurisée.....	40
Annexes.....	42
A1 - Architecture du projet.....	43
A2 - Diagrammes Merise.....	44
A3 - Captures d'écran.....	45
A4 - Documentation API Swagger.....	46
A5 - Pipeline CI/CD.....	48
A6 - Tests.....	49

Présentation du projet

Description

Dkbldr est une application web mobile-first dédiée à la création et à la gestion de decks de cartes pour le jeu *Magic: The Gathering*.

Le projet vise à simplifier le processus de deckbuilding en proposant une interface intuitive, connectée à la base de données de cartes de *Scryfall*.

L'application permet aux utilisateurs de créer, modifier et partager des decks. Il intègre également une dimension sociale, avec un système de suivi (followers/following) d'utilisateurs, de recommandations personnalisées et de statistiques sur les archétypes les plus populaires.

Ce projet répond au besoin des utilisateurs d'avoir une application de deckbuilding mobile-first, accessible à chaque instant. Tout le monde n'a pas accès à une interface desktop, laquelle est majoritaire chez les autres sites similaires (*Scryfall*, *Moxfield*, *Archidekt*). L'objectif numéro 1 de ce projet est de résoudre ce problème.

Périmètre du projet

Le projet comprend :

- Une **API Node.js avec Express**, connectée à **Supabase** pour l'authentification, la gestion des utilisateurs et le stockage des données, et connectée à l'**API de Scryfall** pour les données des cartes.
- Une **application front-end React** (PWA) optimisée mobile, construite avec **Vite**.
- Des fonctionnalités principales : création et édition de decks, recherche et ajout de cartes, gestion de la collection personnelle, import/export de decks, likes, commentaires, recommandations personnalisées.
- Des services secondaires : hébergement d'images (avatars, bannières de profil), affichage des decks récents/populaires, et analytics sociaux.

L'application est conçue pour être déployée sur Railway via des conteneurs **Docker** distincts pour l'API et pour l'app.

Cibles adressées

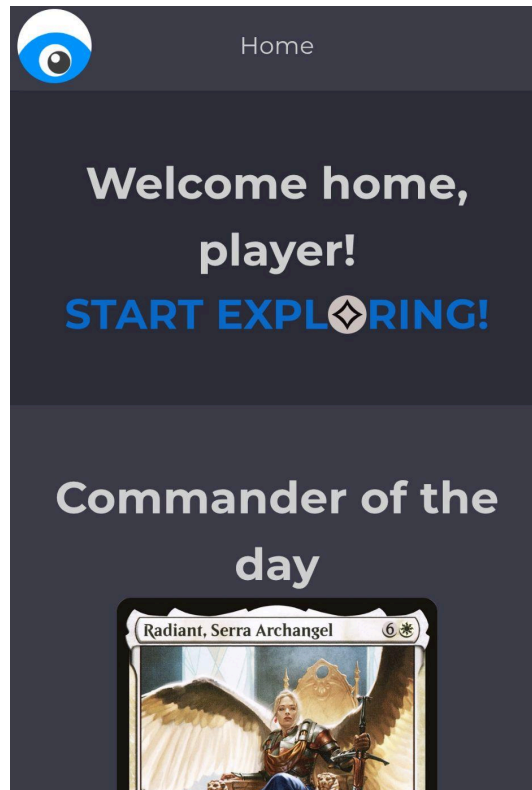
Le projet s'adresse principalement :

- Aux **joueurs de Magic: The Gathering** souhaitant créer, organiser et tester leurs decks.
- Aux **communautés en ligne** de joueurs, grâce aux fonctionnalités sociales (suivi, partages, likes).
- Aux **collectionneurs**, qui peuvent marquer les cartes qu'ils possèdent et filtrer leurs decks en fonction de leur collection réelle.

Arborescence de l'application

L'arborescence fonctionnelle de l'application est la suivante :

- **Page d'accueil**
 - "Commander of the day"
 - Derniers decks des personnes suivies
 - Recommandations personnalisées



- **Page de connexion / inscription**
 - Simple formulaire
- **Page profil utilisateur**
 - Informations personnelles (avatar, description, followers/following)
 - Decks créés
- **Page deck**
 - Vue générale du deck (commanders, couleurs, statistiques)
 - Mainboard & Sideboard
 - Options et outils (import/export, édition, partage, commentaires)
- **Page de recherche**
 - Plusieurs options de recherche (utilisateurs, decks, commanders...)

Cahier des charges

Dans l'optique de la conception du projet, un cahier des charges à été rédigé.

Dénomination

- “**Magic**” se réfère au jeu de cartes *Magic: The Gathering*.
- “**Joueur**” se réfère aux utilisateurs de l'application.
- “**Deck**” se réfère à un assemblage de différentes cartes destinées à être jouées. Le procédé d'assemblage est nommé “**Deckbuilding**”.

Introduction

Contexte et Objectifs du Projet

Le projet vise à développer une application web mobile pour les joueurs de Magic, leur permettant de construire des decks et de socialiser avec d'autres joueurs depuis leur téléphone. L'objectif principal est d'offrir une alternative mobile-first aux autres sites web similaires qui n'offrent pas cette expérience utilisateur.

Public Cible

Cette application s'adresse à un public cible majoritaire :

- **Joueurs**: Les joueurs de Magic.
- **Collectionneurs**: Les collectionneurs de cartes

Objectifs du Projet

Objectifs Généraux

- Faciliter la création de decks pour les joueurs n'ayant pas accès aux outils web similaires.
- Permettre aux joueurs de gérer et de cataloguer leur collection.
- Permettre aux joueurs de s'inspirer de decks d'autres utilisateurs, sur la même plateforme.

Bénéfices Attendus

- Gain de temps et accès facilité pour les joueurs.
- Meilleure gestion des decks, ainsi que de la collection personnelle des joueurs.
- Meilleure indication des decks populaires, ainsi que des meilleures cartes dans le domaine.

Spécifications Fonctionnelles

Gestion des decks

- **Création et Gestion de decks:** Les utilisateurs peuvent créer, modifier et supprimer des decks.
- **Affichage des decks:** Les autres utilisateurs peuvent consulter les decks, et les évaluer.

Outil de Recherche

- **Recherche de carte:** Dans le cadre du deckbuilding, les utilisateurs peuvent utiliser une barre de recherche dans l'éditeur pour ajouter des cartes à leurs decks.
- **Recherche de deck :** Les utilisateurs peuvent utiliser la barre de recherche générale pour trouver des decks par leur nom.
- **Recherche d'utilisateurs :** Les utilisateurs peuvent utiliser la barre de recherche générale pour trouver des utilisateurs par leur nom.
- **Recherche de "Commandant" :** Les utilisateurs peuvent utiliser la barre de recherche générale pour trouver des decks en fonction de leur "Commandant".

Import/Export de decks

- Fonctionnalité d'import/export de decks en un simple fichier texte copié dans le presse-papier.

Spécifications Techniques

Technologies Utilisées

- **Front-End :** Développement en React + Vite.
- **Back-End :** API interne développée avec Express et [Node.js](https://nodejs.org/).
- **Déploiement :** CI/CD mettant en relation Github Actions, Docker et Railway

Contraintes Techniques

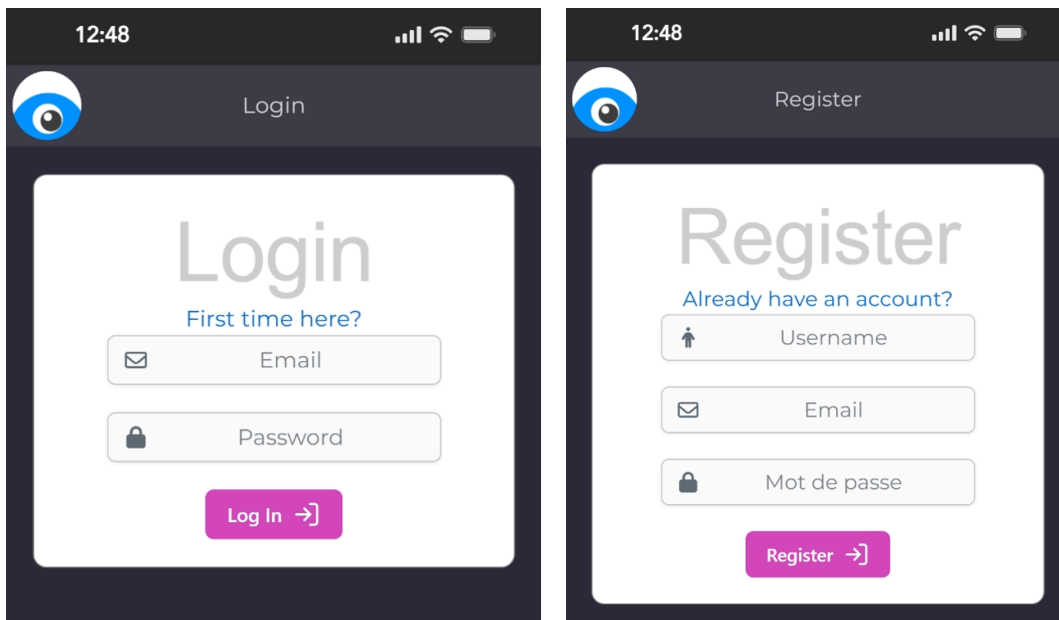
- Étant donné que l'application est mobile-first, il faut que l'app soit légère et les requêtes minimisées.
- L'application devra éventuellement héberger une quantité croissante d'utilisateurs et de données. L'accent sera mis sur la scalabilité.

Descriptif des fonctionnalités

Authentification

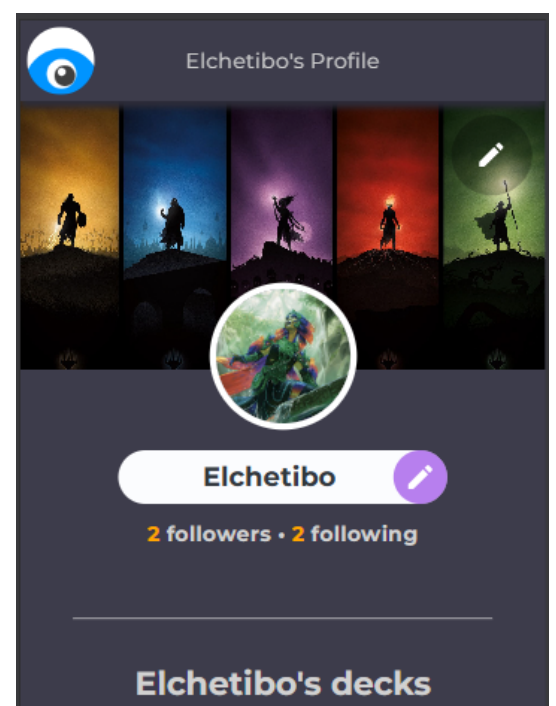
Les utilisateurs peuvent accéder aux formulaires d'inscription et de connexion sur les pages “/register” et “/login” respectivement.

L'authentification est gérée par Supabase qui possède un système sécurisé, où l'administrateur peut lui-même choisir les modes d'authentification, en sélectionnant des “providers”, des services qui permettent de valider l'authentification. Pour ce projet, j'ai choisi d'utiliser un provider d'email. L'utilisateur nouvellement inscrit reçoit un email lui invitant à valider son compte en cliquant sur le lien. Le mot de passe est automatiquement haché.



Espace “profil”

Une fois connectés, les utilisateurs peuvent accéder à leur page de profil depuis la navbar révélée par le bouton central. Sur cette page, ils peuvent modifier leur nom d'utilisateur, leur bannière et image de profil. Une liste de leurs decks construits est également affichée, et les utilisateurs peuvent en créer de nouveaux à partir



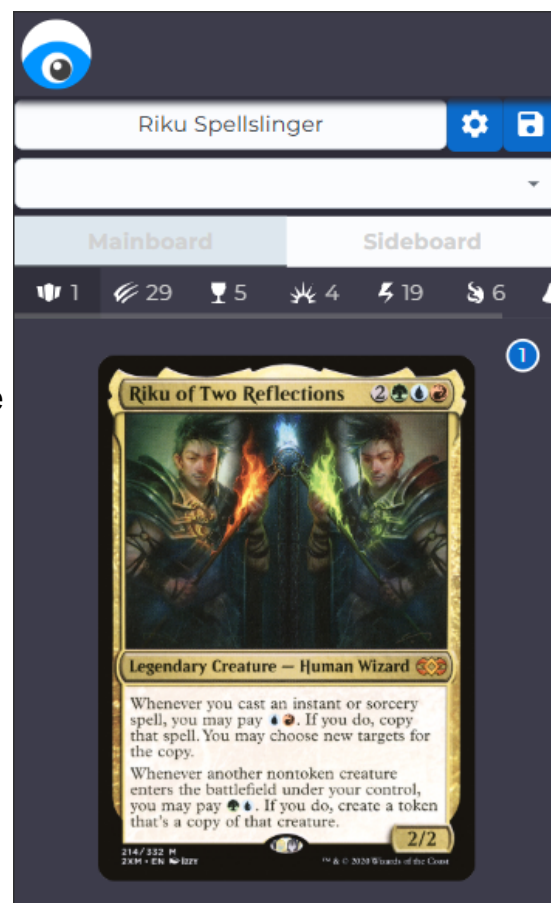
d'un bouton dédié, ou cliquer sur un de leurs decks existants pour le modifier.

Construction de deck (“Deckbuilding”)

Deckbuilding

La page de deckbuilding montre un champ de texte pour nommer son deck, une barre de recherche connectée à l'API externe de Scryfall (qui répertorie toutes les cartes de Magic, ainsi que les données associées) pour ajouter des cartes, deux onglets “Mainboard” (le deck principal) et “Sideboard” (les cartes qui pourraient être intéressantes), un onglet pour chaque type de carte dans le deck (Créature, Artefact, Enchantement, Éphémère, Rituel, Planeswalker, Bataille et Terrain sont les 8 types existants à ce jour), et chacun de ces onglets montrent la liste des cartes appartenant à ces catégories.

En cliquant sur une carte, les utilisateurs peuvent en modifier la quantité, ou voir sa fiche sur Scryfall.

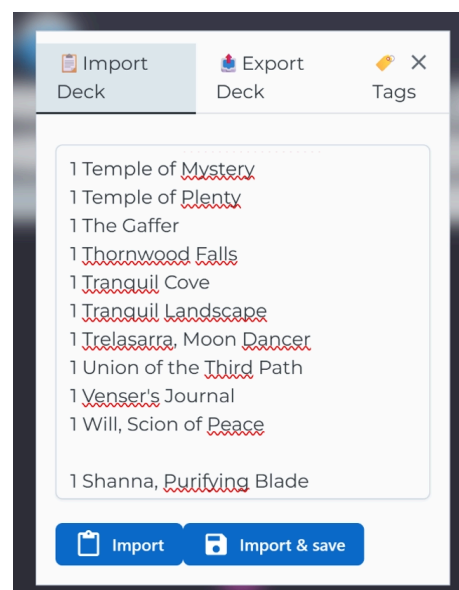


Outils & options

Il existe également un bouton qui permet d'importer/exporter le deck au format texte standardisé dans la forme suivante :

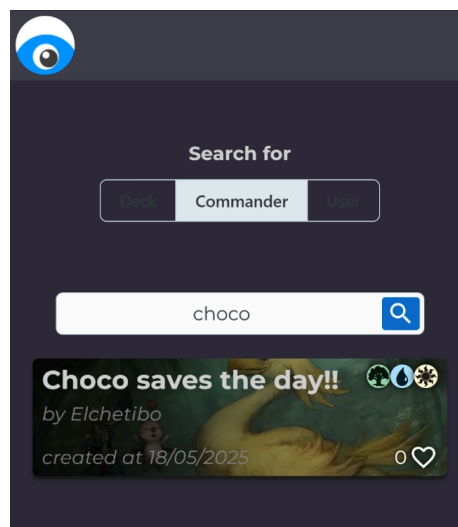
```
<qte> <nom_de_la_carte>
```

Dans les options se trouve également la possibilité de sauvegarder son deck, ainsi que d'y ajouter des “tags” représentant chacun des archétypes de jeu populaires.



Page de recherche

La page de recherche affiche une barre de recherche ainsi que plusieurs filtres de recherche (chercher un Commandant, Utilisateur, Deck). L'utilisateur peut entrer le nom correspondant à sa recherche désirée, et une liste de résultats apparaît.



Decks recommandés

Cette section est affichée sur la page d'accueil. Il s'agit d'une petite liste de decks recommandés en fonction des tags, ou des couleurs récemment utilisées par l'utilisateur.

Decks d'abonnements

Cette section est affichée sur la page d'accueil. Il s'agit d'une petite liste de decks récemment créés par les utilisateurs auxquels l'utilisateur actuel est abonné.

“Commander of the Day”

Cette section est affichée sur la page d'accueil. Il s'agit d'une carte de Commandant sélectionnée au hasard, mais propre à chaque utilisateur. Cette carte change une fois toutes les 24 heures, à minuit (fuseau horaire UTC+2).



Home

Welcome home, player!
START EXPLORING!

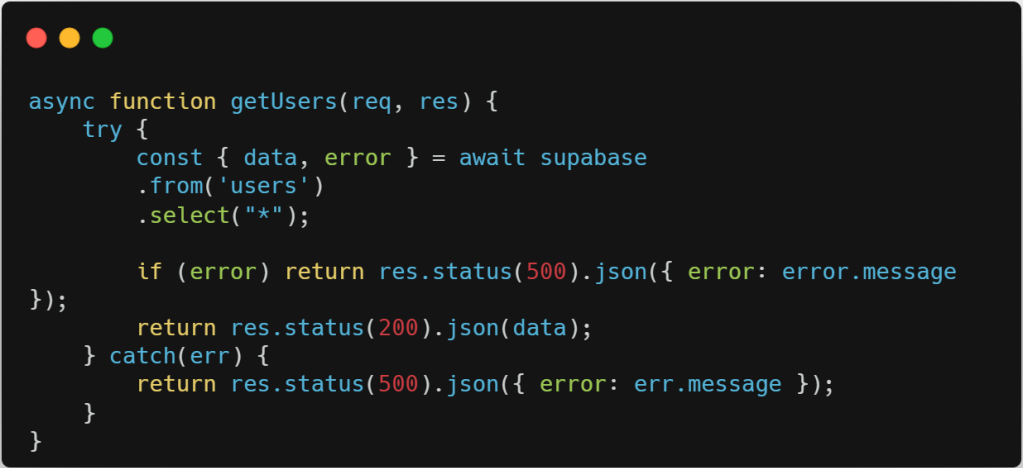
Commander of the day



Spécifications techniques

Technologies back-end

Le back-end repose sur Node.js avec le framework Express, choisi pour sa simplicité, sa légèreté et sa flexibilité dans la création d'API REST. Express permet de définir facilement les routes, de gérer les requêtes HTTP et d'intégrer différents middlewares tels que la gestion du CORS, le traitement des données JSON ou encore la documentation de l'API.



```
async function getUsers(req, res) {
  try {
    const { data, error } = await supabase
      .from('users')
      .select("*");

    if (error) return res.status(500).json({ error: error.message });

    return res.status(200).json(data);
  } catch(err) {
    return res.status(500).json({ error: err.message });
  }
}
```

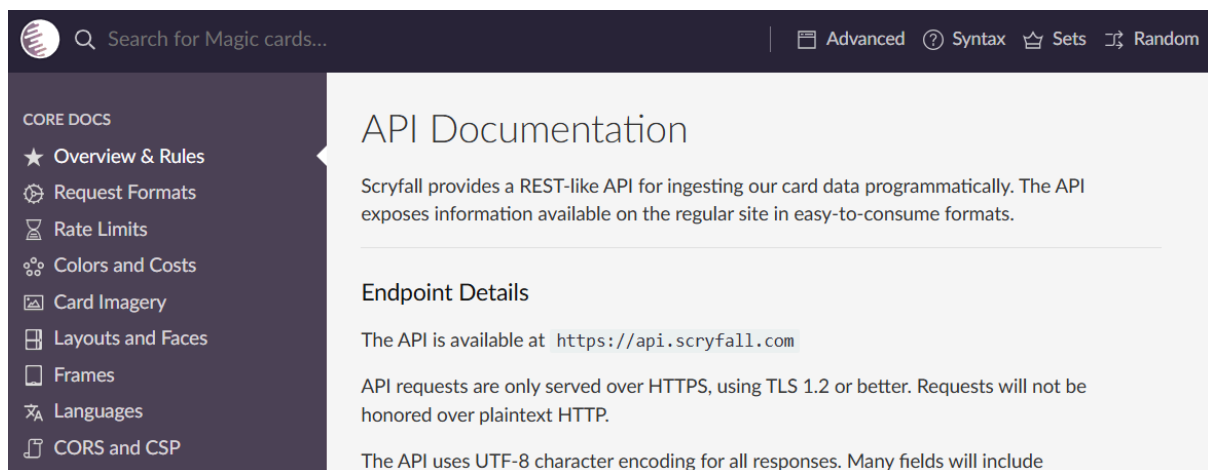
Middleware [Express.js](#) pour récupérer les utilisateurs. Notez l'utilisation du module Supabase.

L'architecture du projet est organisée en plusieurs couches distinctes (routes, contrôleurs et services) afin de respecter le principe de séparation des responsabilités. Les routes définissent les différents points d'entrée de l'API, les contrôleurs assurent le traitement des requêtes et la génération des réponses HTTP, tandis que les services encapsulent la logique métier ainsi que les interactions avec les sources de données. Cette organisation facilite la maintenance du code, son évolution et l'écriture de tests unitaires.

La persistance des données est assurée par une base de données PostgreSQL, hébergée sur Supabase. Cette solution offre une base relationnelle performante tout en proposant des services complémentaires tels que l'authentification des utilisateurs, la gestion des autorisations (Row Level Security) et un espace de stockage de fichiers utilisé pour les avatars et les bannières de profil. Les échanges avec la base de données sont réalisés au moyen du client JavaScript fourni par Supabase, permettant d'effectuer les opérations CRUD de manière sécurisée.

Le back-end communique également avec l'API publique Scryfall afin de récupérer en temps réel les informations détaillées sur les cartes Magic: The Gathering. Les données obtenues sont ensuite exploitées par les différents services de l'application, évitant ainsi de stocker localement une base de données volumineuse tout en bénéficiant d'informations constamment mises à jour.

Les tests automatisés sont réalisés avec Vitest, qui permet de vérifier le bon fonctionnement des routes, des contrôleurs et des services. Les tests sont exécutés aussi bien en environnement local que dans la chaîne d'intégration continue, garantissant la stabilité de l'application avant chaque déploiement.



Documentation de l'API Scryfall

Enfin, la documentation de l'API est générée avec Swagger (OpenAPI). Celle-ci décrit l'ensemble des endpoints disponibles, les paramètres attendus ainsi que les réponses possibles, facilitant ainsi les échanges entre le front-end et le back-end ainsi que la maintenance et les évolutions futures du projet.

Technologies front-end

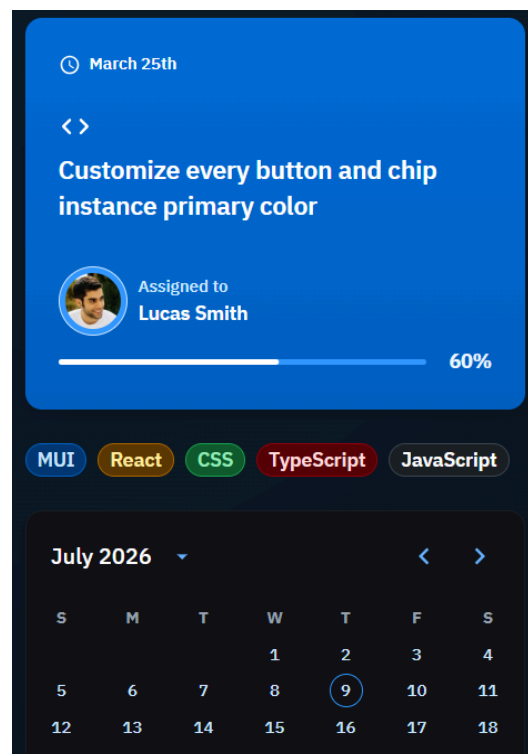
Le front-end est développé avec **React**, associé à **Vite** comme outil de développement et de build :

- **React** a été retenu pour son approche basée sur les composants réutilisables, qui facilite la création d'interfaces complexes tout en améliorant la maintenabilité du code. Son écosystème riche et sa forte adoption dans le développement web moderne en font également un choix pérenne.
- **Vite** a été privilégié en raison de sa rapidité de démarrage, de son rechargement instantané des modules (*Hot Module Replacement*) et de ses performances lors de la génération des versions de production, offrant ainsi une expérience de développement plus fluide que les outils plus anciens.

L'interface utilisateur est construite avec **Joy UI**, une bibliothèque de composants développée par MUI.

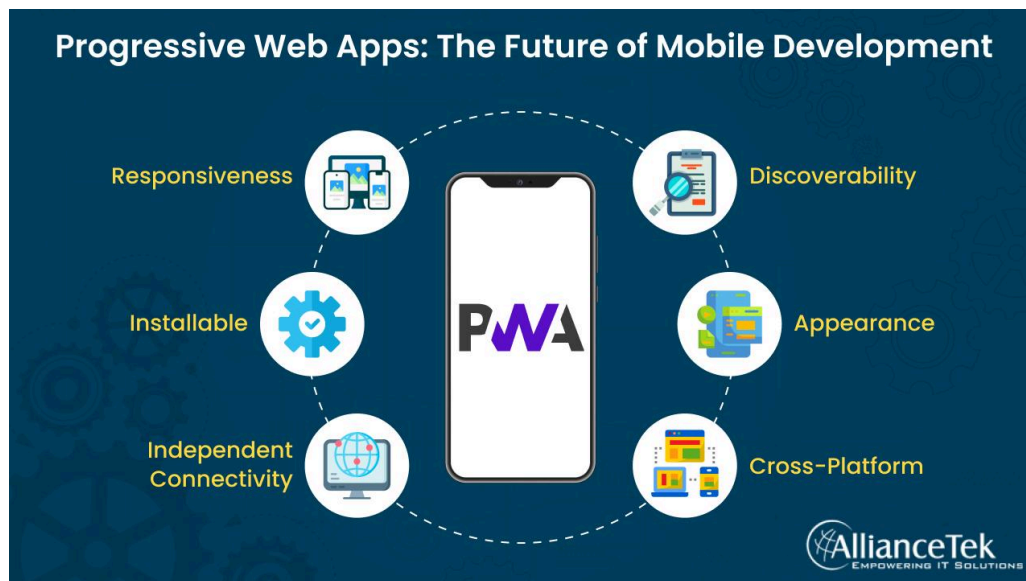
Ce choix permet de disposer de composants modernes, accessibles et personnalisables tout en conservant une identité visuelle homogène sur l'ensemble de l'application.

Joy UI simplifie également la mise en œuvre du responsive design grâce à son système de mise en page et à ses composants adaptatifs, ce qui a facilité l'adoption d'une approche **mobile-first**. Cette approche garantit une expérience utilisateur optimale sur smartphone tout en s'adaptant naturellement aux écrans de plus grande taille.



Exemples de composants créés avec JoyUI

L'application est conçue comme une **Progressive Web App (PWA)** afin d'offrir une expérience proche de celle d'une application native. Grâce au manifeste de l'application et au service worker, les ressources statiques peuvent être mises en cache, permettant de réduire les temps de chargement, d'améliorer les performances lors des visites suivantes et d'assurer le fonctionnement de certaines fonctionnalités en mode hors ligne. L'utilisateur peut également installer l'application directement sur son appareil depuis son navigateur, sans passer par une boutique d'applications.



L'architecture du front-end est organisée autour d'une séparation claire des responsabilités. Les composants React sont principalement chargés de l'affichage et des interactions avec l'utilisateur, tandis que les échanges avec les services externes sont regroupés dans des services dédiés. Cette organisation limite le couplage entre l'interface graphique et la logique d'accès aux données, facilitant ainsi les évolutions, les tests et la réutilisation du code.

La communication avec le back-end est réalisée via des requêtes HTTP vers l'API Express, tandis que certaines fonctionnalités, comme l'authentification des utilisateurs ou la gestion des fichiers, utilisent directement les services proposés par **Supabase**. La centralisation de ces appels dans des services dédiés permet de simplifier les composants React et d'assurer une gestion cohérente des erreurs, des réponses de l'API et des traitements asynchrones dans l'ensemble de l'application.

Environnement de développement

Planification et Conception

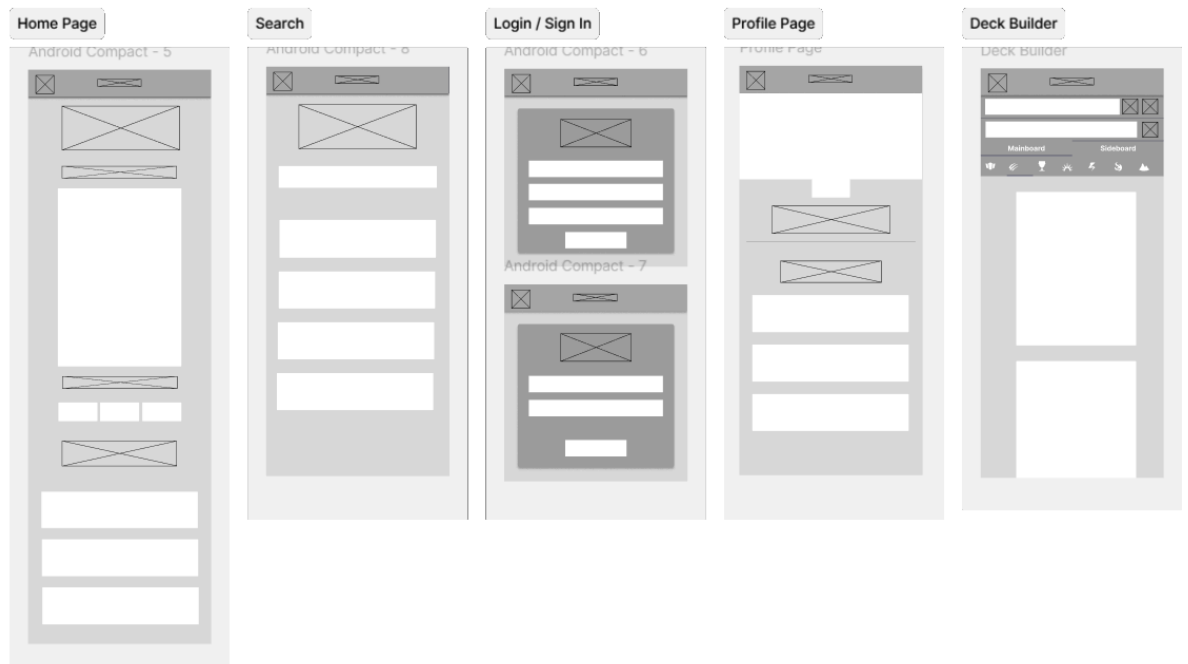
La conception du projet a commencé avec l'écriture d'un cahier des charges, suivi par l'établissement d'un tableau Kanban créé sur **Trello**. Ce tableau permet la visualisation et la gestion des différentes tâches du projet. Chaque tâche est représentée par un ticket assignable à n'importe quel membre du projet. Le tableau contient plusieurs colonnes pour classer l'état des tâches :

- **Terminé** : les tâches de cette colonne sont terminées
- **A faire** : les tâches n'ont pas encore été activement travaillées
- **A tester** : les tâches dont l'accomplissement est présumément terminé, mais ont besoin d'être testées pour définitivement les valider
- **En cours** : les tâches en cours de production

Lorsque le développement d'une tâche change, il suffit juste de déplacer le ticket associé dans la colonne appropriée pour signaler l'avancement de cette tâche.

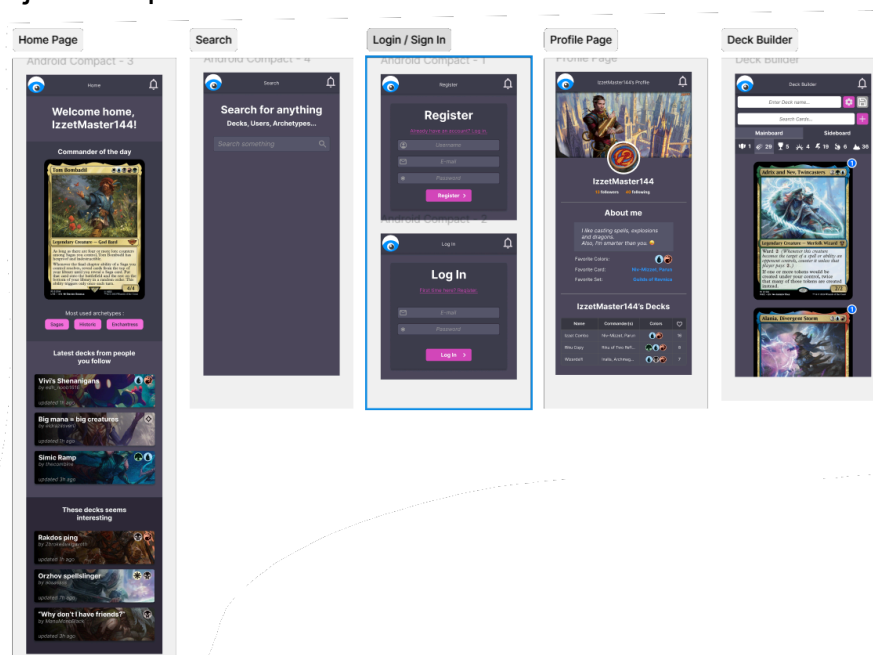
Ensuite vient la conception du design de l'application sur **Figma**, un outil en ligne permettant de conceptualiser la structure visuelle de l'application, à plusieurs stades de développement :

1. **Zoning** : Les "zones" de l'application (structure basique, placement des sections, etc) y sont représentées. Pas de couleur ni de mise en forme, que des formes abstraites.
2. **Wireframe** : Reprends les zones du stade précédent pour y ajouter plus de contexte (telle zone contient un bouton, telle zone une image, telle zone un texte, etc). Encore une fois, ça reste très sobre.



Exemple du wireframe

3. **Prototype** : Ajoute enfin de la couleur (selon la charte graphique préalablement décidée) et du texte aux endroits appropriés. Un prototype est également navigable si l'outil utilisé le permet. Heureusement, Figma dispose d'un système de prototypage basique mais fonctionnel (navigation, interactions, etc)
4. **Haute-fidélité** : La phase finale de la maquette. Elle doit être la référence en terme d'esthétique, de structure et de fonctionnalité. Tout y est ajouté et perfectionné.



Outils de développement

Le développement du projet a été réalisé avec **Visual Studio Code**, choisi pour sa légèreté, sa richesse fonctionnelle et son important écosystème d'extensions. Cet environnement facilite le développement JavaScript grâce à l'autocomplétion, au débogage intégré, au formatage automatique du code et à l'intégration native avec Git.

Contrôle de version

Le code source est versionné avec **Git** et hébergé sur **GitHub**, permettant de suivre l'historique des modifications, de travailler sur des branches dédiées aux nouvelles fonctionnalités et de faciliter la collaboration entre développeurs.

Git utilise un système de branches. Chaque branche représente une "version" du code source. Pour toute fonctionnalité implémentée dans l'application (aussi bien au niveau de l'interface utilisateur que du back-end), une branche est créée pour isoler ladite fonctionnalité.

La structure de branches du repository se base sur deux branches :

- **main** : contient uniquement le code validé et prêt au déploiement
- **dev** : la branche de code principale, contient du code stable et est utilisée pour créer de nouvelles branches.

Conteneurisation

L'application est entièrement **conteneurisée avec Docker**. Des images distinctes sont construites pour le front-end et le back-end, ce qui permet d'isoler les différents services de l'application et d'assurer un fonctionnement identique quel que soit l'environnement d'exécution. Cette approche élimine les problèmes liés aux différences de configuration entre les postes de développement, les serveurs d'intégration continue et les environnements de production. Docker simplifie également le déploiement sur des plateformes cloud compatibles avec les conteneurs.

Pipeline CI/CD

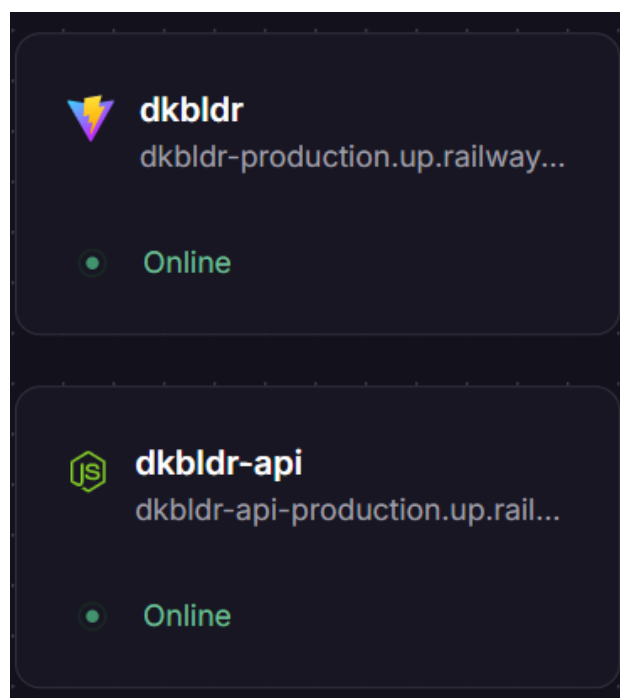
L'intégration et le déploiement continus (CI/CD) sont assurés par **GitHub Actions**. À chaque modification envoyée sur le dépôt, un workflow

automatise l'installation des dépendances, l'exécution des tests automatisés avec Vitest, la construction des images Docker puis leur déploiement sur Railway. Cette automatisation réduit les risques d'erreurs humaines, garantit que le code respecte les critères de qualité définis par le projet et accélère la mise en production des nouvelles versions.

Déploiement

Le déploiement est réalisé sur **Railway**, une plateforme cloud permettant d'héberger les différents services de l'application.

Railway prend en charge le déploiement automatique des conteneurs Docker, la gestion des variables d'environnement, l'exposition des services sur Internet ainsi que la surveillance de leur état de fonctionnement. Cette solution permet de disposer rapidement d'environnements de test et de production sans avoir à administrer un serveur.



L'ensemble de ces outils contribue à mettre en place un environnement de développement moderne, reproductible et automatisé. Ils facilitent le travail quotidien des développeurs, améliorent la qualité du code produit et sécurisent les différentes étapes allant du développement jusqu'au déploiement en production.

Architecture du projet

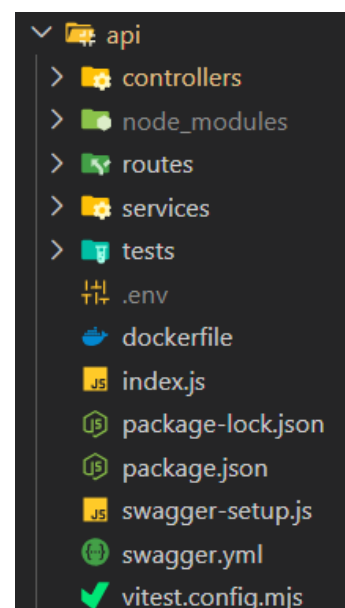
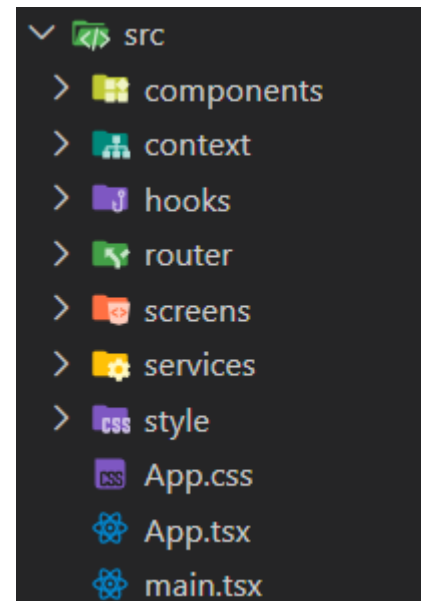
Le projet est organisé selon une architecture en deux parties clairement distinctes : le **front-end**, chargé de l'interface utilisateur, et le **back-end**, responsable de la logique métier et de l'accès aux données. Cette séparation permet à chaque partie de l'application d'évoluer indépendamment, tout en facilitant la maintenance et les tests.

Le **front-end**, situé à la racine du projet, est développé avec React et adopte une organisation par responsabilités. Les composants constituent les éléments de l'interface utilisateur et sont conçus pour être réutilisables dans plusieurs pages de l'application.

Les pages représentent les différentes vues accessibles à l'utilisateur, tandis que les contextes React permettent de partager un état global, comme les informations relatives à l'utilisateur connecté ou les préférences de l'application, sans avoir à propager ces données entre tous les composants. Enfin, les services regroupent l'ensemble des appels au back-end et à Supabase, ce qui isole la logique d'accès aux données du code d'affichage.

Le **back-end**, situé dans le dossier `api/`, suit une architecture en couches inspirée du modèle MVC. Les **routes** définissent les différents endpoints de l'API REST et assurent le routage des requêtes HTTP. Elles délèguent ensuite le traitement aux **contrôleurs**, qui analysent les requêtes, effectuent les vérifications nécessaires et construisent les réponses renvoyées au client.

Les contrôleurs s'appuient sur des **services**, chargés d'implémenter la logique métier ainsi que les interactions avec les différentes sources de données, notamment la base PostgreSQL hébergée sur Supabase ou les API externes telles que Scryfall. Cette organisation limite les



dépendances entre les différentes couches et facilite l'évolution de l'application.

La communication entre le front-end et le back-end s'effectue exclusivement au travers d'une API REST.

Pourquoi cette architecture ?

Cette approche découple totalement l'interface utilisateur de la logique métier : le front-end n'accède jamais directement aux données sensibles et toutes les opérations passent par des endpoints sécurisés.

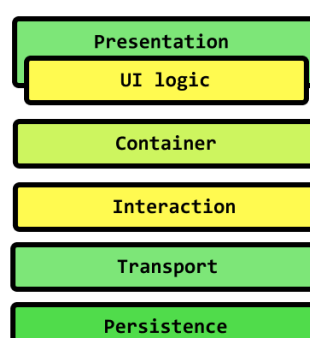
Une telle architecture rend possible le développement futur d'autres clients (application mobile, client desktop ou API publique) sans modifier le fonctionnement interne du serveur.

Supabase occupe une place centrale dans cette architecture. Au-delà de l'hébergement de la base de données PostgreSQL, la plateforme prend en charge l'authentification des utilisateurs, la gestion des autorisations d'accès aux données ainsi que le stockage des fichiers (avatars et bannières de profil).

Le recours à une plateforme Backend-as-a-Service (BaaS) a permis de réduire le temps de développement en s'appuyant sur des services éprouvés, tout en conservant la maîtrise de la logique métier au sein du back-end Express.

Cette organisation respecte le principe de séparation des responsabilités (*Separation of Concerns*), selon lequel chaque couche possède un rôle clairement défini. Elle améliore la lisibilité du code, facilite les tests unitaires et d'intégration, simplifie les opérations de maintenance et permet d'ajouter de nouvelles fonctionnalités avec un impact limité sur le reste de l'application. Cette modularité constitue également un atout pour le travail collaboratif, plusieurs développeurs pouvant intervenir simultanément sur différentes parties du projet sans générer de conflits importants.

Separation of Concerns **Each layer handles a separate concern**



L'architecture du projet s'appuie sur plusieurs principes de conception inspirés de **SOLID**. Les responsabilités sont réparties entre les composants React, les contrôleurs de l'API, les services métier et la couche d'accès aux données, ce qui favorise la lisibilité, la maintenabilité et l'évolutivité de l'application.

Les principes liés à l'héritage et aux interfaces sont en revanche moins présents, le projet étant développé en JavaScript, un langage privilégiant davantage la composition de modules que l'héritage orienté objet.

Réalisation

Charte graphique

L'identité visuelle de l'application repose sur une esthétique moderne et minimaliste, axée "dark mode" pour apaiser le regard de l'utilisateur et adaptée à une utilisation mobile-first. Les couleurs principales sont :

- **Rose (#D349BC)** pour les actions et éléments interactifs.
- Gris foncé (**#3D3C4B**) en arrière-plan pour favoriser la lisibilité.
- **Noir/anthracite (#D1D1D1)** pour le texte principal.

La typographie utilisée est Montserrat, garantissant une bonne lisibilité sur tous les écrans. Les icônes suivent une ligne épurée et un design "flat".

Maquette

Les maquettes ont été conçues avec Figma, afin de tester rapidement l'ergonomie de l'application. Le parcours utilisateur est structuré en deux axes:

1. **Exploration** : page d'accueil avec suggestions de decks, recherche par commandeur, onglets de navigation.
2. **Construction** : interface de deckbuilding avec tri par type de carte, menu contextuel sur chaque carte et gestion mainboard/sideboard.

Ces maquettes ont permis de valider les choix UX avant le développement et de garantir une cohérence graphique entre les différentes pages.

Pour les besoins du projet, trois maquettes ont été conçues :

1. **Zoning** : permet de visualiser les zones des pages de la webapp
2. **Wireframe** : permet de structurer les éléments de base, et de les visualiser leur agencement
3. **Haute-fidélité** : maquette se basant sur le wireframe, permettant de visualiser le rendu final de l'application

Conception de la base de données

La base de données repose sur PostgreSQL via Supabase. Les principales tables sont :

- **users** : informations sur les utilisateurs (id, email, username, avatar_url...).

- **decks** : données des decks créés (id, owner_id, nom, format, date, cards).
- **archetypes** : archétypes de jeu représentés par des tags dans l'application ("Aggro", "Kindred", "Burn", "Midrange", etc)

Cette structure permet de gérer efficacement les relations tout en assurant la scalabilité (ajout futur de statistiques, favoris, commentaires, etc.)

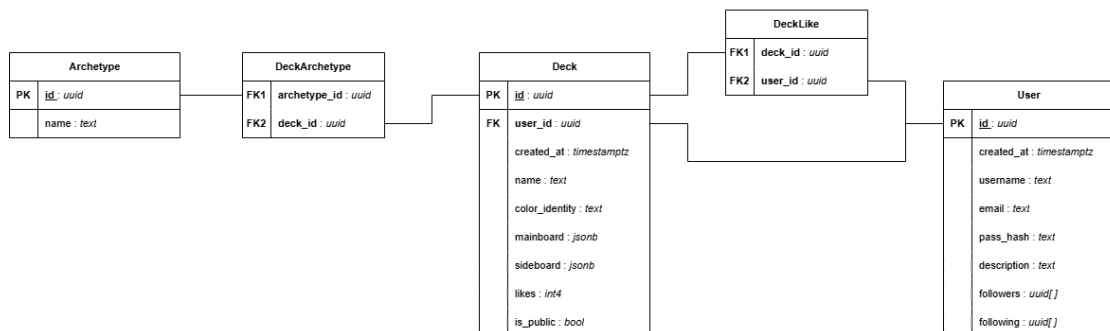
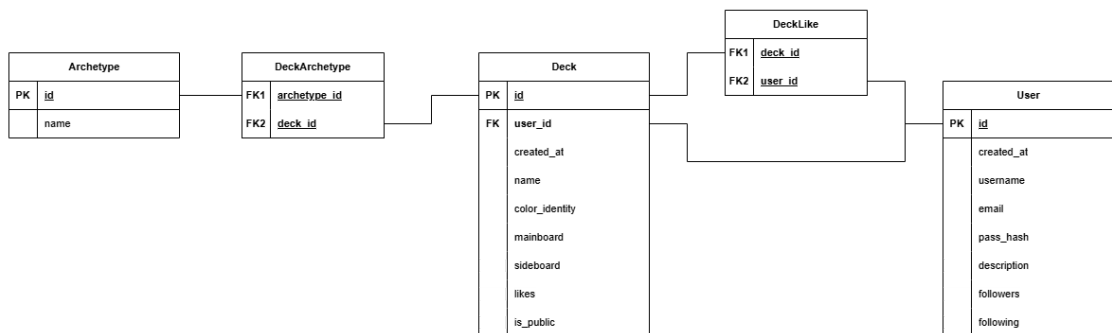
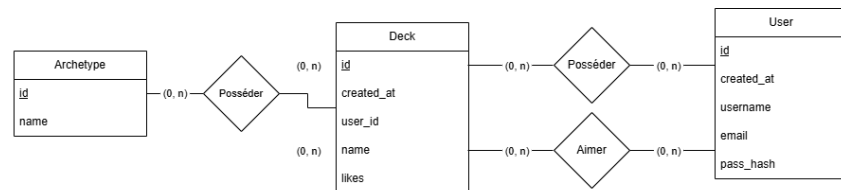
La structure de cette base de données à été conçue en utilisant la méthode Merise.

Ci-dessous, de haut en bas:

1. *Modèle Conceptuel de Données*

2. *Modèle Logique de Données*

3. *Modèle Physique de Données*



Authentification

L'authentification est gérée par Supabase Auth, basé sur les JWT (JSON Web Tokens). Les utilisateurs peuvent se connecter via email/mot de passe. Les tokens JWT sont stockés dans `sessionStorage` afin de limiter les risques liés au XSS.

Supabase Auth permet de gérer l'authentification et les sessions des utilisateurs : inscription, connexion, réinitialisation des mots de passe (via email), MFA/2FA et même la connexion via providers (compte Google, Github, etc) ou OAuth. Ensuite, les JWT fournis sont utilisés dans les requêtes, avec les politiques RLS.

La méthode de validation d'inscription actuelle est une solution mail SMTP, qui envoie un e-mail à l'adresse renseignée lors de l'inscription pour confirmer la création du compte.

Cette approche sans état permet une gestion sécurisée et performante des sessions, tout en restant simple à intégrer côté front-end.

Stockage de fichiers

Le stockage des fichiers (avatars, bannière de profil) est assuré par Supabase Storage. Les fichiers sont organisés par dossier/bucket (`avatars/`, `decks/`) et chaque ressource est liée à son propriétaire via un champ `user_id`.

Supabase Storage est un système permettant de stocker des fichiers (image, texte, vidéo, etc) avec une intégration forte à Postgres et aux politiques RLS.

Ce système se base sur un modèle de "buckets":

- Bucket : un "dossier" logique
- Object : un fichier stocké dans un bucket (avec un chemin, comme dans une arborescence de fichiers)
- Le stockage est géré via les tables Postgres, comme par exemple `storage.buckets` ou `storage.objects` ; mais le développeur interagit surtout avec les API fournies en tant que couche d'abstraction.

Les règles de sécurité Supabase (Row-Level-Security Policies) permettent de restreindre l'accès :

- Un utilisateur ne peut lire/écrire que ses propres fichiers.

Sources des icônes

Les icônes utilisées proviennent principalement de :

- La bibliothèque FontAwesome, fournie par JoyUI
- Le site Flaticon, répertoriant des icônes gratuites.

Compétences attestées

Développer une application sécurisée :

- **Installer et configurer son environnement de travail en fonction du projet**

Pour le projet **Dkbldr**, l'environnement de travail a été mis en place en fonction des besoins spécifiques liés à une application web moderne.

- Le code du projet a été écrit sur **Visual Studio Code**, avec des extensions appropriées.
- Le **back-end** est développé avec **Node.js** et **Express**, accompagné de contrôleurs sécurisés pour gérer les requêtes.
- Le **front-end** repose sur **React** avec **Vite** pour optimiser le développement et le rendu.
- Les deux parties de l'application sont contenues dans des environnements **Docker**, facilitant la portabilité et la cohérence entre le développement local et la production.
- L'authentification et la gestion des données sont assurées via **Supabase**, offrant une couche de sécurité intégrée avec gestion des utilisateurs et JWT.

- **Développer des interfaces utilisateur**

Le front-end a été conçu en tant que **PWA mobile-first** avec une interface claire et intuitive :

- Affichage des decks répartis en onglets (mainboard, sidebar).
- Navigation simplifiée avec un système de Tabs sticky et un menu contextuel pour chaque carte.
- Placeholders et squelettes de chargement pour améliorer l'expérience utilisateur.
- Design basé sur des composants réutilisables (JoyUI).

- **Développer des composants métier**

Le coeur de l'application repose sur des composants robustes :

- **Gestion des decks** : création, modification, suppression, import/export au format standard
- **Gestion des utilisateurs** : suivi (follower/following), profils personnalisables et système de recommandation
- **Stockage Supabase** : pour la gestion des données utilisateurs

- **Contribuer à la gestion d'un projet informatique**

La gestion de projet à été assurée via :

- L'utilisation de Git et Github pour le versionning, les branches et les revues de code
- Une pipeline CI/CD Github Actions avec exécution automatique des tests via Vitest
- Une documentation structurée de l'API grâce à Swagger
- Une logique modulaire avec séparation des contrôleurs, services et routes.

Concevoir et développer une application sécurisée organisée en couches :

- **Analyser les besoins et maquetter une application**

Le projet a été amorcé par une analyse des besoins des joueurs de Magic, avec un focus sur :

- **Une interface mobile-first** : adaptée aux besoins en déplacement (tournois, parties amicales)
- **L'accessibilité** : de nos jours, tout le monde possède un smartphone, mais tout le monde n'a pas accès à une interface desktop, malheureusement majoritaire dans les autres outils de ce type.
- **Les fonctionnalités sociales** : pour encourager les interactions entre joueurs.

- Définir l'architecture logicielle d'une application

L'architecture suit une organisation en couches claires :

- **Front-end (React + Vite)** : affichage et interactions utilisateur
- **API (Express)** : contrôleurs et services séparés, logique métier centralisée
- **Base de données (Supabase + Postgres)** : persistance des données relationnelles
- **Stockage objet (Supabase Storage)** : fichiers utilisateurs (avatars, bannières)

- Concevoir et mettre en place une base de données relationnelle

Pour répondre aux besoins fonctionnels de l'application, une base de données relationnelle a été mise en place en **PostgreSQL via Supabase**. Le choix de cette technologie repose sur sa robustesse, sa conformité ACID et la possibilité d'assurer l'intégrité des données tout en bénéficiant d'outils modernes (API REST générées, authentification intégrée).

La conception du schéma relationnel s'est appuyée sur une modélisation des principales entités du projet :

- **Utilisateurs (users)** : identifiés par un UUID, avec des informations de profil (pseudo, email, avatar, bannière, description).
- **Decks (decks)** : chaque deck appartient à un utilisateur, possède un nom, une identité couleur et un ensemble de cartes réparties en mainboard/sideboard.
- **Archétypes (archetypes)** : tags permettant de classer et de rechercher les decks.
- **Relations sociales** : tables de jointure permettant de gérer les **followers/following**, les **likes** et les interactions sociales entre utilisateurs.

L'intégrité des données est assurée par :

- L'utilisation de **clés primaires** (UUID générés automatiquement par Supabase).

- Des **clés étrangères** pour assurer la cohérence (exemple : **user_id** dans **decks**).
- Des contraintes (unicité des emails, relations cascade lors de la suppression d'un utilisateur).

Cette structuration garantit une base solide, capable de gérer à la fois des **relations complexes** (réseau social) et des **volumes importants de données** (listes de cartes).

- Développer des composants d'accès aux données SQL et NoSQL

L'application combine deux modes de gestion de données :

1. Accès SQL via Supabase (Postgres) :

Les opérations CRUD classiques sont implémentées dans des contrôleurs, les séparant des routes auxquelles elles sont associées.

Cette séparation en couche permet une meilleure testabilité et renforce la sécurité en limitant les accès directs à la base.

2. Stockage de fichier (NoSQL-like)

Pour les données non structurées (images de profil et bannières), un stockage objet est utilisé.

- Les fichiers sont envoyés via des requêtes HTTP multi-part en utilisant **Multer**.
- Supabase Storage attribue un chemin unique et génère une URL publique
 - "user/avatar_<timestamp>.png" pour les avatars
 - "user/header_<timestamp>.png" pour les bannières
- L'URL est stockée dans la table "**users**", liant ainsi données SQL et fichiers binaires

Préparer le déploiement d'une application sécurisée

- Préparer et exécuter les plans de test d'une application

Afin d'assurer la qualité et la robustesse de l'application, un processus de tests automatisés a été mis en place

- **Tests Unitaires avec Vitest** sur l'API Express. Ces derniers valident les fonctions critiques.
- Mise en place d'un **environnement temporaire dédié aux tests** en base Supabase : des utilisateurs factices sont créés dans `"beforeAll"`, puis supprimés dans `"afterAll"` une fois les tests effectués.
- Des tests ont également été mis en place pour **vérifier la validité des données** récupérées de Scryfall.
- **Gestion des erreurs vérifiées** : codes HTTP 400 (Requête invalide), 404 (Données introuvables), 500 (Erreur serveur) sont couverts en cas de tests.

Cette démarche garantit que l'évolution de l'API ne cassent pas le fonctionnement existant. Les tests constituent une preuve de sécurité fonctionnelle, et renforcent la confiance avant chaque déploiement.

- Préparer et documenter le déploiement d'une application

Le déploiement a été conçu pour être automatisé et répétable.

- **Dockerisation complète du projet** : deux Dockerfiles distincts (un pour l'application et l'autre pour l'API) permettent de packager les services avec toutes leurs dépendances.
- **Docker Compose** est utilisé en local pour build les deux images en même temps et lancer l'ensemble.
- **Railway** a été choisi comme plateforme de déploiement.

Cette préparation méthodique permet à tout développeur rejoignant le projet de lancer l'application dans un environnement neuf en quelques minutes seulement.

- Contribuer à la mise en production dans une démarche DevOps

Une pipeline CI/CD avec Github Actions a été intégrée afin de sécuriser et automatiser la mise en production :

1. **Phase de test** : à chaque push sur la branche 'main', les tests unitaires sont lancés automatiquement
2. **Phase de build** : si les tests passent, les images Docker de l'API et de l'application sont build et push vers Docker Hub avec un nouveau tag
3. **Phase de déploiement** : Railway est notifié et redéploie automatiquement les conteneurs avec le dernier tag validé.

Cette approche garantit que :

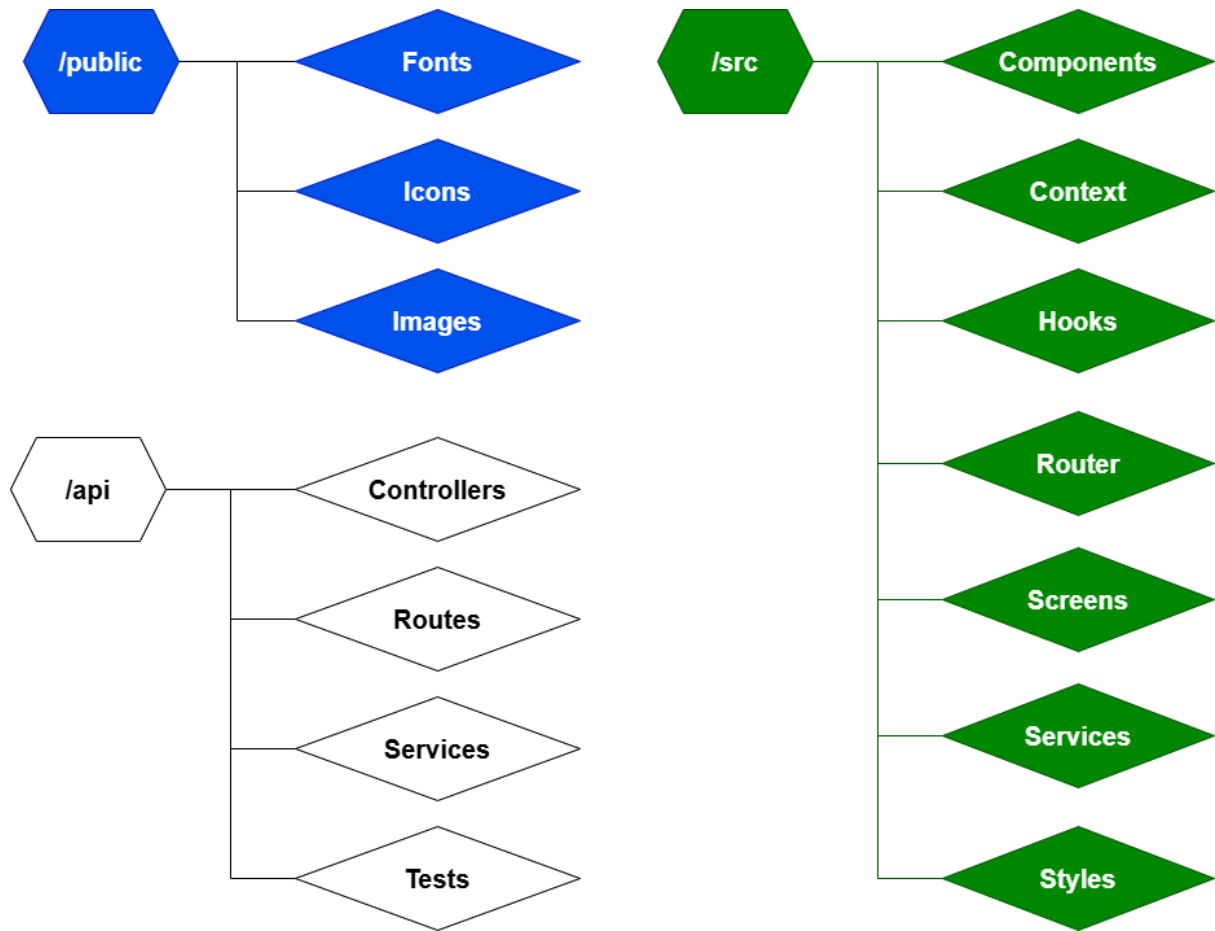
- Aucune version non-testée ne parte en production
- Le déploiement est rapide, reproductible et ne nécessite pas de maintenance manuelle
- La séparation des services (API et front-end) permet une scalabilité et une résilience future.

En appliquant cette démarche DevOps, le projet Dkbldr atteint un niveau de professionnalisme digne d'applications en production réelle :

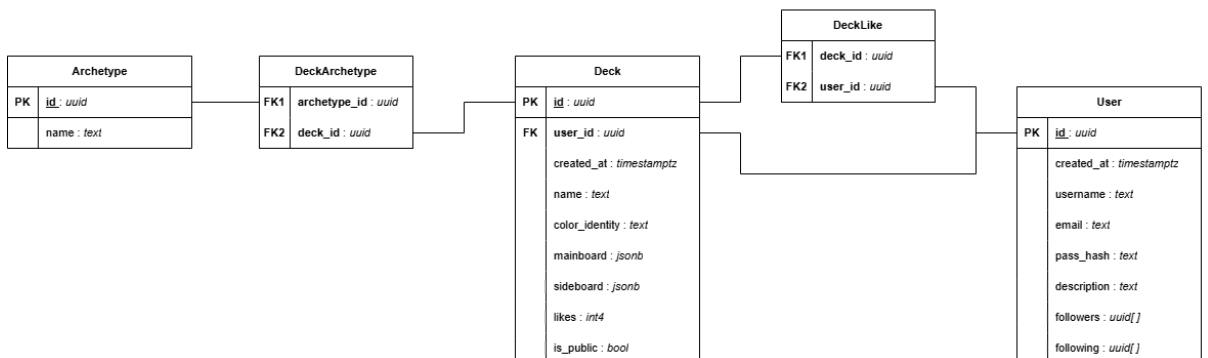
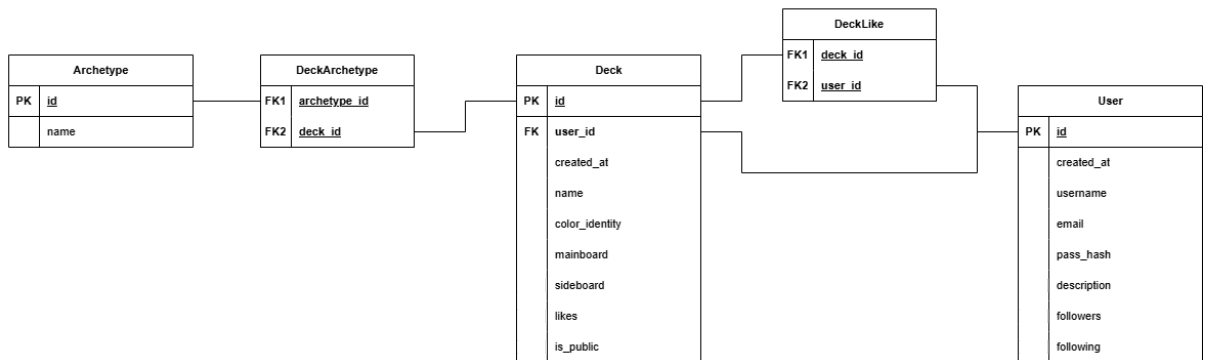
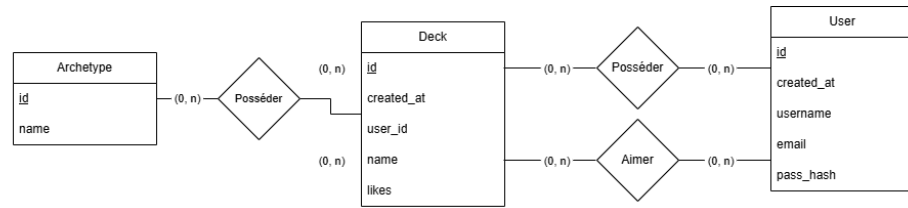
tests => intégration => déploiement => disponibilité continue

Annexes

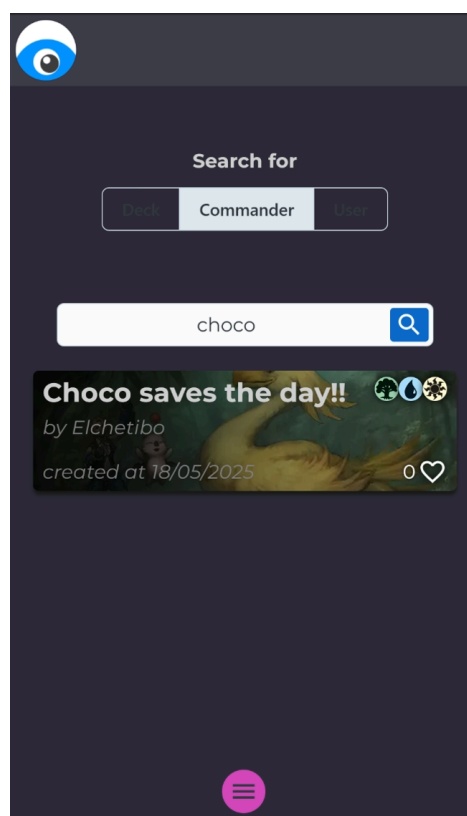
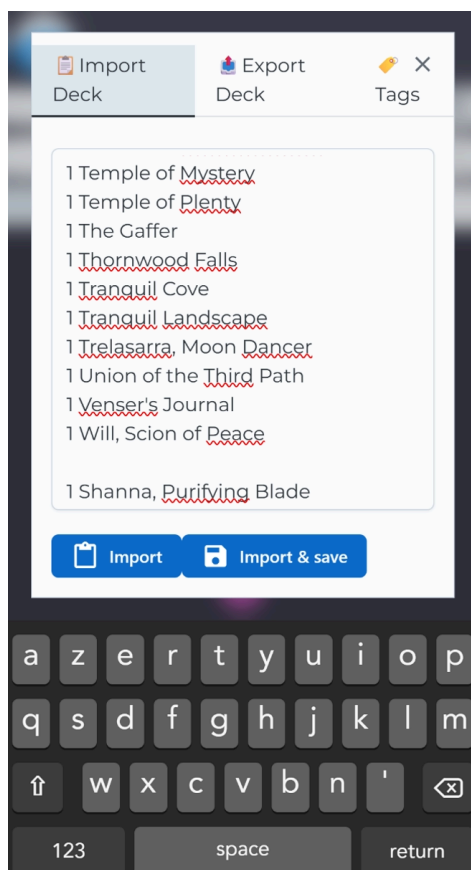
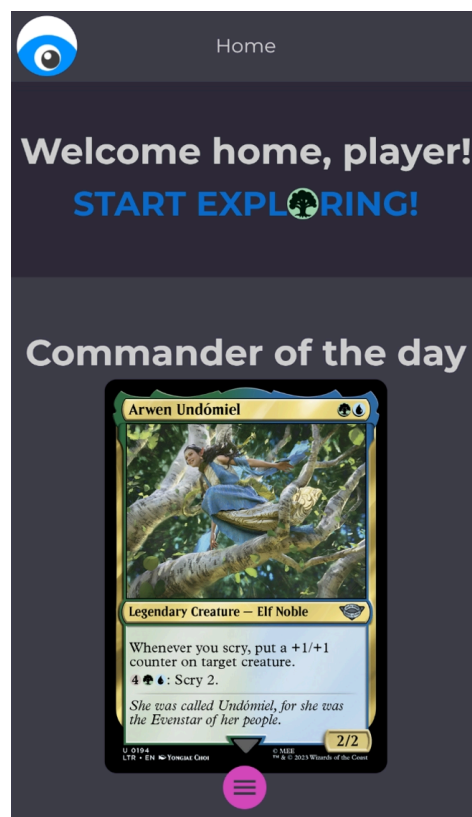
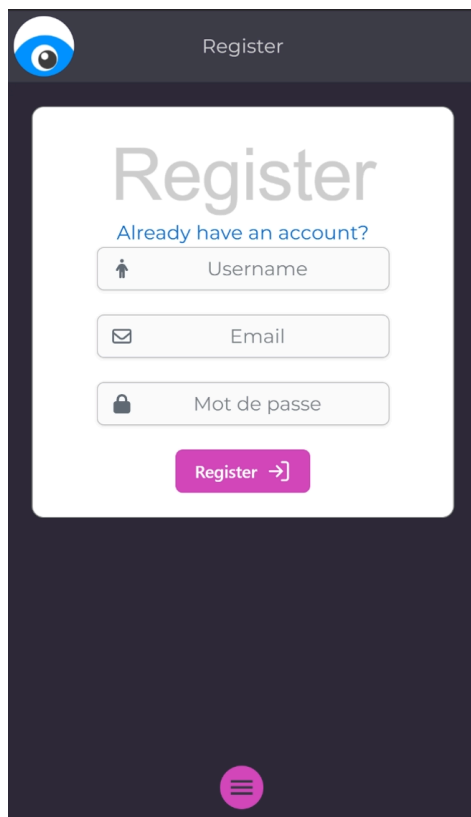
A1 - Architecture du projet



A2 - Diagrammes Merise



A3 - Captures d'écran



A4 - Documentation API Swagger

Decks			^
GET	/api/decks/{deckId}	Get one specific deck by its UUID	▼
PATCH	/api/decks/{deckId}	Updates an existing deck	▼
DELETE	/api/decks/{deckId}	Delete a deck by its UUID	▼
GET	/api/decks/user/{userId}	Get all decks from a specific user	▼
POST	/api/decks/user/{userId}	Create a new deck for a user	▼
Users			^
POST	/api/users/	Create a new user	▼
GET	/api/users/all	Get all users	▼
GET	/api/users/{userId}	Get a user by ID	▼
PATCH	/api/users/{userId}	Update a user	▼
DELETE	/api/users/{userId}	Delete a user	▼
POST	/api/users/follow	Follow a user	▼
POST	/api/users/unfollow	Unfollow a user	▼
Cards			^
GET	/api/cards/random	Get a random Magic The Gathering card	▼
GET	/api/cards/random/edh	Get a random Commander EDH legal commander	▼
POST	/api/cards/search/{q}	Search for a card by name	▼
GET	/api/cards/{cardId}	Get a specific card by its Scryfall ID	▼
GET	/api/cards/{cardId}/prints	Get all prints versions of a specific card	▼

Archetypes



GET

/api/tags/all Get all available archetypes/tags



GET

/api/tags/decks/{deckId} Get all archetypes/tags assigned to a specific deck



POST

/api/tags/deck/{deckId} Add archetypes/tags to a deck



Storage



POST

/api/storage/avatar/{userId} Upload a profile picture for a user



PATCH

/api/storage/avatar/{userId} Update user's profile picture URL in database



POST

/api/storage/header/{userId} Upload a header background image for a user



PATCH

/api/storage/header/{userId} Update user's header background image URL in database



A5 - Pipeline CI/CD

```
name: CI/CD Pipeline

on:
  push:
    branches:
      - "main"
  pull_request:
    branches:
      - "main"

jobs:
  test:
    runs-on: ubuntu-latest
    environment: "production"
    defaults:
      run:
        working-directory: ./api
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 24

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm run test
        env:
          SUPABASE_URL: ${ secrets.SUPABASE_URL }
          SUPABASE_SECRET_KEY: ${ secrets.SUPABASE_SECRET_KEY }

  deploy:
    runs-on: ubuntu-latest
    environment: "production"
    needs: test
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }

      - name: Build & Push API image
        run: |
          docker build -t thibaultkine/dkbldr-api:latest ./api
          docker push thibaultkine/dkbldr-api:latest

      - name: Build & Push App image
        run: |
          docker build \
            --build-arg VITE_SUPABASE_URL=${ secrets.VITE_SUPABASE_URL } \
            --build-arg VITE_SUPABASE_PUBLIC_KEY=${ secrets.VITE_SUPABASE_PUBLIC_KEY } \
            --build-arg VITE_API_URL=${ secrets.VITE_API_URL } \
            -t thibaultkine/dkbldr-app:latest ./

          docker push thibaultkine/dkbldr-app:latest
```


Triggered via push 2 days ago

thibault-kine pushed -o 68eb776 main

Status: Success

Total duration: 1m 30s

Artifacts: -

deploy.yml
on: push

test 14s

deploy 1m 10s

A6 - Tests

```
13 RUN v3.2.7 /home/runner/work/dkbldr/dkbldr/api
14
15 ✓ tests/cards.test.js (2 tests) 204ms
16 stdout | tests/users.test.js > Users Controller - CRUD Operations > createUser > should create a user successfully
17 ● createUser - 201
18
19 stdout | tests/users.test.js > Users Controller - CRUD Operations > updateUser > should update a user successfully
20 ● updateUser - 200
21
22 stdout | tests/users.test.js > Users Controller - CRUD Operations > deleteUser > should delete a user successfully
23 ● deleteUser - 200
24
25 ✓ tests/users.test.js (6 tests) 3084ms
26   ✓ Users Controller - CRUD Operations > createUser > should create a user successfully 1488ms
27   ✓ Users Controller - CRUD Operations > updateUser > should update a user successfully 532ms
28   ✓ Users Controller - CRUD Operations > deleteUser > should return 500 if user ID is invalid 477ms
29
30 Test Files 2 passed (2)
31 Tests 8 passed (8)
32 Start at 12:34:59
33 Duration 3.45s (transform 75ms, setup 0ms, collect 126ms, tests 3.29s, environment 0ms, prepare 159ms)
```